

数据结构 完整复习资料（含模拟试题与速记手册）

子冉一人 整理

2026 年 7 月 12 日

目录

第一章 绪论	4
知识点总结	4
习题	5
第二章 线性表	10
知识点总结	10
习题	11
第三章 栈和队列	16
知识点总结	16
习题	17
第四章 字符串和多维数组	21
知识点总结	21
习题	22
第五章 树和二叉树	26
知识点总结	26
习题	28
第六章 图	32
知识点总结	32
习题	33
第七章 查找技术	37
知识点总结	37
习题	38
第八章 排序技术	42
知识点总结	42

子冉一人	3
习题	43
附录 A 习题答案与解析	48
附录 B 模拟试题	67
附录 C 考前速记手册	71

第1章 绪论

本章知识点总结

一、问题求解与程序设计

程序设计的一般过程：问题分析 → 想法（抽象模型）→ 算法（数据表示 + 数据处理）→ 程序（形式化）。

核心公式（Wirth 公式）：

$$\text{算法} + \text{数据结构} = \text{程序}$$

计算思维三阶段：

1. 模型化（抽象思维）——建立数据模型
2. 逻辑思维——设计算法步骤
3. 形式化——编程实现

二、数据结构的基本概念

核心术语：

- 数据：能输入计算机并被程序识别和处理的符号集合
- 数据元素：数据的基本单位，整体处理
- 数据项：构成数据元素的最小单位

数据结构的定义： $\text{Data_Structure} = (D, R)$ ， D 为数据元素有限集， R 为 D 上关系的集合。

逻辑结构分类：

类型	关系	示例
集合	无关系	数学集合
线性结构	一对一	线性表、栈、队列
树结构	一对多	文件系统
图结构	多对多	交通网络

存储结构：

- 顺序存储：连续的存储单元，逻辑关系由位置表示
- 链接存储：任意的存储单元，逻辑关系由指针表示

抽象数据类型 (ADT)：一个数据模型以及定义在该模型上的一组操作。只考虑逻辑结构和操作接口，不考虑具体实现。

三、算法的基本概念

算法定义：对特定问题求解步骤的描述，指令的有限序列。

五大特性：有穷性、确定性、可行性、输入、输出。

好算法特性：正确性、健壮性、可理解性、抽象分级（ 7 ± 2 原则）、高效性。

算法描述方法：自然语言、流程图、伪代码（推荐）、程序语言。

四、算法分析

时间复杂度：

$$T(n) = O(f(n))$$

常见复杂度比较：

$$O(1) < O(\log_2 n) < O(n) < O(n \log_2 n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$$

非递归算法分析：从内层向外展开，求和。

递归算法分析：通用分治递推式 $T(n) = aT(n/b) + cn^k$ 。

三种情况：最好情况、最坏情况（实时系统必分析）、平均情况。

空间复杂度： $S(n) = O(f(n))$ ，关注辅助空间。

本章习题

（一）选择题

1. 数据结构在计算机内存中的表示是指（ ）。
 - A. 数据的逻辑结构
 - A. 数据的存储结构
 - A. 数据的运算
 - A. 数据元素之间的关系
2. 以下哪一个不是算法的特性？（ ）

- A. 有穷性
 - A. 确定性
 - A. 可读性
 - A. 可行性
3. 以下时间复杂度中，哪一个增长最慢？（ ）
- A. $O(n)$
 - A. $O(\log_2 n)$
 - A. $O(n^2)$
 - A. $O(n \log_2 n)$
4. 抽象数据类型（ADT）不包括以下哪个要素？（ ）
- A. 数据模型
 - A. 操作接口
 - A. 实现细节
 - A. 基本操作
5. 线性结构的数据元素之间存在（ ）关系。
- A. 一对多
 - A. 一对一
 - A. 多对多
 - A. 无关系
6. 以下算法描述方法中，推荐用于描述算法思想的是（ ）。
- A. 自然语言
 - A. 流程图
 - A. 伪代码
 - A. 程序语言
7. 算法的时间复杂度取决于（ ）。
- A. 问题的规模
 - A. 计算机的运算速度
 - A. 程序设计语言

A. 编译器的质量

8. 以下关于好算法的说法，错误的是（ ）。

A. 正确性——对合法输入得出正确结果

A. 健壮性——能抵抗非法输入

A. 高效性——运行时间短、辅助空间少

A. 越复杂越好

9. 算法分析的目的是（ ）。

A. 找出数据结构的合理性

A. 研究算法中的输入/输出关系

A. 分析算法的效率以求改进

A. 分析算法的可读性

10. 在顺序表中，数据元素之间的逻辑关系由（ ）表示。

A. 指针

A. 存储位置

A. 下标

A. 关键码

（二）辨析判断题

11. 算法可以用自然语言、流程图、伪代码或程序语言来描述。

12. 数据的逻辑结构是指数据在计算机内存中的存储方式。

13. 抽象数据类型（ADT）只考虑数据的逻辑结构和基本操作，不考虑实现细节。

14. 算法必须包含至少一个输入。

15. $O(2^n)$ 的时间复杂度属于多项式时间，是易于求解的问题。

16. 顺序存储结构和链接存储结构都可以表示线性关系。

17. 算法的健壮性是指算法能够正确解决问题。

18. 好的算法应遵循米勒原则（ 7 ± 2 ），即合适的抽象分级。

19. 数据的存储结构实质上是内存分配的问题。

20. 递归算法的时间复杂度通常用递推关系式来分析。

（三）简答题

21. 简述数据结构的逻辑结构和存储结构之间的关系。

22. 什么是算法的有穷性？举例说明什么样的操作步骤不满足有穷性。

23. 简述大 O 记号的定义及其在算法分析中的作用。

（四）编程题

24. 用 C++ 伪代码编写一个函数，计算一维整型数组 $A[n]$ 中所有元素之和，并分析其时间复杂度。

25. 编写一个 C++ 函数，实现在顺序表中按值查找某个元素，返回其位序（下标）。表中元素类型为 `int`，顺序表结构自行定义。

26. 已知单链表结点结构如下，编写函数统计单链表中结点个数。

```
1 struct Node {  
2     int data;  
3     Node* next;  
4 };
```

第2章 线性表

本章知识点总结

一、线性表的逻辑结构

定义： n ($n \geq 0$) 个类型相同的数据元素组成的有限序列：

$$(a_1, a_2, \dots, a_i, \dots, a_n)$$

特点：同一性、有穷性、有序性（一对一线性关系）。

基本操作：初始化、销毁、判空、求长度、遍历、查找（按位/按值）、插入、删除。

二、线性表的顺序存储——顺序表

```
1 const int MaxSize = 100;
2 template <class DataType>
3 class SeqList {
4     DataType data[MaxSize];
5     int length;
6 };
```

特点：随机存取 ($O(1)$)，存储密度高，插入/删除需移动元素 ($O(n)$)，需预先分配存储空间。

插入：平均移动 $n/2$ 个元素， $O(n)$ 。删除：平均移动 $(n-1)/2$ 个元素， $O(n)$ 。

三、线性表的链接存储——单链表

结点结构：

```
1 template <class DataType>
2 struct Node {
3     DataType data;
4     Node<DataType>* next;
5 };
```

特点：动态存储，逻辑次序与物理次序不一定相同，插入/删除只需修改指针 ($O(1)$)，查找需遍历 ($O(n)$)。

建表方法：头插法（逆序）、尾插法（正序）。

四、循环链表与双向链表

循环链表：尾结点指向头结点（或首结点），可从任意位置遍历。

双向链表：

```
1 template <class DataType>
2 struct DNode {
3     DataType data;
4     DNode<DataType> *prior, *next;
5 };
```

插入/删除需同时修改 prior 和 next 指针。

五、顺序表与链表的比较

比较维度	顺序表	链表
存储空间	静态分配	动态分配
存取方式	随机存取	顺序存取
插入/删除	$O(n)$ （需移动元素）	$O(1)$ （已知位置）
存储密度	高	低
适用场景	频繁查找	频繁插入删除

本章习题

（一）选择题

1. 线性表采用顺序存储结构时，其数据元素的地址（ ）。
A. 一定连续
A. 部分连续，部分不连续
A. 一定不连续
A. 连续与否均可
2. 单链表中，已知指针 p 指向某结点，要在其后插入新结点 s ，正确的语句是（ ）。
A. $s \rightarrow \text{next} = p \rightarrow \text{next}; p \rightarrow \text{next} = s;$
A. $p \rightarrow \text{next} = s; s \rightarrow \text{next} = p;$
A. $s \rightarrow \text{next} = p; p \rightarrow \text{next} = s;$

- A. $p \rightarrow \text{next} = s \rightarrow \text{next}; s \rightarrow \text{next} = p;$
3. 在长度为 n 的顺序表中插入一个元素，平均需要移动 () 个元素。
- A. n
- A. $n/2$
- A. $(n-1)/2$
- A. $(n+1)/2$
4. 线性表若采用链式存储结构，要求内存中可用存储单元的地址 ()。
- A. 必须是连续的
- A. 部分连续部分不连续
- A. 一定是不连续的
- A. 连续与否均可
5. 设一个链表最常用的操作是在表尾插入和删除元素，则选用 () 最省时间。
- A. 单链表
- A. 带头结点的单循环链表
- A. 双向链表
- A. 带尾指针的循环单链表
6. 顺序表中，数据元素的存储地址是其下标的 () 函数。
- A. 线性
- A. 非线性
- A. 对数
- A. 指数
7. 双向链表中，在 p 所指结点前插入 s 结点的语句序列中，最后一个语句应该是 ()。
- A. $p \rightarrow \text{prior} = s$
- A. $s \rightarrow \text{prior} = p \rightarrow \text{prior}$
- A. $p \rightarrow \text{prior} \rightarrow \text{next} = s$
- A. $s \rightarrow \text{next} = p$
8. 链表中最常用的操作是在最后一个元素之后插入和删除第一个元素，选用 () 最节省时间。

- A. 单链表
- A. 双向链表
- A. 带头结点的单循环链表
- A. 带尾指针的单循环链表

9. 以下关于顺序表的说法，正确的是（ ）。

- A. 插入和删除操作的时间复杂度均为 $O(1)$
- A. 支持随机存取
- A. 存储密度较低
- A. 无需预分配存储空间

10. 以下关于链表的说法，正确的是（ ）。

- A. 链表的存储空间是连续分配的
- A. 链表只能顺序存取
- A. 链表的插入操作需要移动数据元素
- A. 链表比顺序表占用更少的存储空间

（二）辨析判断题

- 11. 线性表的每个元素都有一个直接前驱和一个直接后继。
- 12. 顺序表可以随机存取任一元素。
- 13. 单链表删除操作不需要移动元素，只需修改指针。
- 14. 循环链表中没有空指针。
- 15. 顺序表的存储密度比链表高。
- 16. 在单链表中，插入操作的时间复杂度一定是 $O(1)$ 。
- 17. 双向链表比单链表占用更多的存储空间。
- 18. 线性表的长度是固定的，不能动态变化。
- 19. 顺序表是一种随机存取结构。
- 20. 链表适合频繁插入和删除操作的场景。

（三）简答题

21. 简述顺序表和链表各自的优缺点及适用场景。
22. 什么是头结点？在单链表中设置头结点有什么好处？
23. 比较循环链表和双向链表的区别与联系。

（四）编程题

24. 编写 C++ 函数，在带头结点的单链表中删除所有值为 x 的结点。结点结构为 `Node { int data; Node* next; }`。
25. 编写 C++ 函数，实现顺序表上第 i 个位置插入元素 e 的操作。

-
26. 编写 C++ 函数，实现两个有序单链表的合并（非递减顺序），要求利用原结点空间。

第3章 栈和队列

本章知识点总结

一、栈

定义：限定仅在表尾（栈顶）进行插入和删除操作的线性表。核心特性：**后进先出（LIFO）**。

基本操作：Push（入栈）、Pop（出栈）、GetTop（取栈顶）、Empty（判空）。

顺序栈：

- 用一维数组存储， $top = -1$ 为空
- Push: $data[++top] = x$
- Pop: $x = data[top--]$

链栈：

- 以单链表头结点为栈顶，入栈用头插法
- 无需判满（动态分配）

应用：括号匹配、进制转换、函数嵌套调用、表达式求值。

二、队列

定义：只允许在队尾插入、队头删除的线性表。核心特性：**先进先出（FIFO）**。

基本操作：EnQueue（入队）、DeQueue（出队）、GetQueue（取队头）、Empty（判空）。

循环队列：

- front 指向队头，rear 指向队尾的下一个位置
- 队空: $front = rear$
- 队满: $(rear + 1) \% QueueSize = front$ （牺牲一个单元）
- 入队: $rear = (rear + 1) \% QueueSize$
- 出队: $front = (front + 1) \% QueueSize$

链队列：带头结点单链表，front 指向头结点，rear 指向队尾结点。

本章习题

（一）选择题

1. 栈和队列的共同点是（ ）。
 - A. 都是先进先出
 - A. 都是后进先出
 - A. 只允许在端点处插入和删除
 - A. 没有共同点
2. 一个栈的入栈序列为 1, 2, 3, 4, 则以下不可能的出栈序列是（ ）。
 - A. 1, 2, 3, 4
 - A. 4, 3, 2, 1
 - A. 3, 4, 2, 1
 - A. 3, 1, 2, 4
3. 循环队列用数组 $Q[0 \dots m-1]$ 存储元素, front 和 rear 的含义如本章所述, 则判空的条件是（ ）。
 - A. $\text{rear} = \text{front}$
 - A. $\text{rear} + 1 = \text{front}$
 - A. $(\text{rear} + 1) \% m = \text{front}$
 - A. $(\text{rear} - 1) \% m = \text{front}$
4. 栈的典型应用不包括（ ）。
 - A. 括号匹配
 - A. 进制转换
 - A. 任务调度
 - A. 函数调用
5. 在链栈中, 进行入栈操作时（ ）。
 - A. 需要判满
 - A. 不需要判满
 - A. 需要判空

- A. 以上都不对
6. 循环队列中，队列长度为 m ，队头指针 $front$ 指向队头元素，队尾指针 $rear$ 指向队尾的下一个位置，则队列中元素个数为（ ）。
- A. $rear - front$
- A. $rear - front + 1$
- A. $(rear - front + m) \% m$
- A. $(rear - front - 1) \% m$
7. 一个栈的入栈序列为 a, b, c, d, e ，则下列不可能的出栈序列是（ ）。
- A. a, b, c, d, e
- A. e, d, c, b, a
- A. d, c, e, b, a
- A. d, e, c, a, b
8. 队列的 FIFO 特性是指（ ）。
- A. 先进队的元素后出队
- A. 后进队的元素先出队
- A. 先进队的元素先出队
- A. 以上都不对
9. 5 个不同的元素入栈，可能得到的不同出栈序列共有（ ）种。
- A. 42
- A. 14
- A. 24
- A. 120
10. 以下关于链栈的说法，正确的是（ ）。
- A. 链栈的入栈和出栈都在栈底进行
- A. 链栈的入栈用头插法
- A. 链栈必须带头结点
- A. 链栈的操作比顺序栈复杂

（二）辨析判断题

11. 栈是一种操作受限的线性表。
12. 栈和队列都是线性结构。
13. 在循环队列中， $\text{front} = \text{rear}$ 时队列已满。
14. 栈的后进先出特性适用于括号匹配问题。
15. 链栈的入栈操作需要判断栈是否已满。
16. 队列的插入操作只能在队尾进行。
17. 循环队列可以避免“假溢出”问题。
18. 顺序栈的入栈操作需要判断栈是否已满。
19. 链队列的出队操作需要判断队列是否为空。
20. 两个栈共享同一段存储空间时，空间利用率比单独分配更高。

（三）简答题

21. 简述栈和队列的区别。
22. 什么是“假溢出”？循环队列如何解决该问题？
23. 举例说明栈在表达式求值中的应用。

24. 用 C++ 实现顺序栈的类定义, 包含 Push 和 Pop 操作。

25. 编写 C++ 函数，利用栈实现括号匹配的检测。假设只有小括号。

26. 用 C++ 实现链队列的入队和出队操作（带头结点）。

第4章 字符串和多维数组

本章知识点总结

一、字符串及其模式匹配

字符串定义：零个或多个字符组成的有限序列。

BF 算法（暴力匹配）：

- 从主串和模式串第一个字符开始依次比较
- 匹配失败，主串回溯下一个位置，模式串从头开始
- 时间复杂度 $O(n \times m)$

KMP 算法：

- 利用已匹配部分信息，主串不回溯
- next 数组：记录模式串的最长相等前后缀长度
- 时间复杂度 $O(n + m)$

二、多维数组

逻辑结构：数组是线性表的推广。一维数组是线性表，二维数组是线性表的线性表。

特点：元素类型相同，不能执行插入/删除，基本操作只有存取和修改。

存储结构：顺序存储。

- 按行优先：先存行号小的
- 按列优先：先存列号小的

寻址公式（按行优先）：

$$\text{Loc}(a_{ij}) = \text{Loc}(a_{l_1 l_2}) + ((i - l_1) \times (h_2 - l_2 + 1) + (j - l_2)) \times c$$

三、特殊矩阵的压缩存储

对称矩阵： $a_{ij} = a_{ji}$ ，只存储下三角， $n^2 \rightarrow n(n+1)/2$ 。

$$k = i \times (i - 1) / 2 + j - 1$$

三角矩阵：只存储下三角或上三角元素。

对角矩阵：只存储非零对角线元素。

本章习题

（一）选择题

1. 字符串是（ ）。
 - A. 有限个字符的序列
 - A. 无限个字符的序列
 - A. 可变长的数据集合
 - A. 只能包含字母的序列
2. BF 模式匹配算法的时间复杂度是（ ）。
 - A. $O(n + m)$
 - A. $O(n \times m)$
 - A. $O(n^2)$
 - A. $O(m^2)$
3. KMP 算法中，next 数组的作用是（ ）。
 - A. 记录模式串的长度
 - A. 记录主串中已匹配的字符数
 - A. 记录模式串的最长相等前后缀长度
 - A. 记录主串的比较位置
4. 二维数组 $A[1..m][1..n]$ 按行优先存储， $A[1][1]$ 的地址为 1000，每个元素占 4 个单元，则 $A[i][j]$ 的地址为（ ）。
 - A. $1000 + ((i - 1) \times n + (j - 1)) \times 4$
 - A. $1000 + ((i - 1) \times m + (j - 1)) \times 4$
 - A. $1000 + (i \times n + j) \times 4$
 - A. $1000 + ((i - 1) \times (j - 1)) \times 4$
5. 特殊矩阵压缩存储的基本思想是（ ）。
 - A. 为每个非零元素单独分配空间

- A. 为值相同的元素只分配一个存储空间
 - A. 只存储对角线元素
 - A. 只存储矩阵的边界元素
6. 将 n 阶对称矩阵压缩存储到一维数组中, 至少需要 () 个单元。
- A. n^2
 - A. $n(n+1)/2$
 - A. $n(n-1)/2$
 - A. n
7. 二维数组按列优先存储时, $A[i][j]$ 前面有 () 列元素。
- A. i
 - A. j
 - A. $j-1$
 - A. $i-1$
8. 空串是指 ()。
- A. 长度为 0 的串
 - A. 只包含空格的串
 - A. 长度为 1 的串
 - A. 不含字母的串
9. KMP 算法相对于 BF 算法的最大优势是 ()。
- A. 实现更简单
 - A. 不需要额外空间
 - A. 主串不需要回溯
 - A. 可以处理任意类型的数据
10. 以下关于数组的叙述, 正确的是 ()。
- A. 数组支持插入和删除操作
 - A. 数组元素的类型可以不同
 - A. 数组是线性表的推广
 - A. 数组的长度可以动态变化

（二）辨析判断题

11. KMP 算法的时间复杂度是 $O(n \times m)$ 。
12. 空串的长度为零。
13. 对称矩阵的压缩存储仅需存储上三角或下三角部分。
14. 数组是线性结构。
15. 数组没有插入和删除操作。
16. BF 算法中，匹配失败时主串指针需要回溯。
17. KMP 算法中，next 数组的值只与模式串有关。
18. 矩阵压缩存储后，无法实现随机存取。
19. 一维数组是线性表。
20. 三角矩阵压缩存储的单元数和对角矩阵相同。

（三）简答题

21. 简述 BF 算法与 KMP 算法的区别与联系。
22. 什么是对称矩阵的压缩存储？给出 n 阶对称矩阵下三角元素 a_{ij} ($i \geq j$) 在一维数组中的下标公式。

23. 简述多维数组按行优先存储的地址计算方式。

(四) 编程题

24. 用 C++ 实现 BF 算法的函数，返回模式串 T 在主串 S 中首次出现的位置（下标从 0 开始）。
25. 编写 C++ 函数，求 n 阶对称矩阵中下三角元素 a_{ij} ($i \geq j$) 在一维数组中的下标 k 。
26. 实现三维数组按行优先的地址计算函数，已知首地址、各维大小和元素字节数，计算指定下标的地址。

第5章 树和二叉树

本章知识点总结

一、树的逻辑结构

定义： n ($n \geq 0$) 个结点的有限集合。 $n = 0$ 为空树。非空树满足：

1. 有且仅有一个根结点
2. 其余结点分为 m ($m \geq 0$) 个互不相交的子树

术语：结点的度、树的度、叶子结点、分支结点、结点的层次、树的深度（高度）、森林。

二、二叉树

定义：二叉树是 n ($n \geq 0$) 个结点的有限集合，或为空，或由一个根结点和两棵互不相交的左右子树组成。

二叉树不是度为 2 的树！二叉树和树是两种不同的树结构。

特点：每个结点最多两棵子树；二叉树是有序的。

特殊二叉树：

- 斜树：左斜树或右斜树
- 满二叉树：所有分支结点都有左右子树，叶子全在同一层
- 完全二叉树：叶子只出现在最下两层，且靠左排列

性质：

1. 第 i 层最多有 2^{i-1} 个结点
2. 深度为 k 的二叉树最多有 $2^k - 1$ 个结点
3. $n_0 = n_2 + 1$ (叶子数 = 度为 2 的结点数 + 1)
4. 完全二叉树深度为 $\lfloor \log_2 n \rfloor + 1$

存储结构：顺序存储（完全二叉树用数组）、链式存储（二叉链表，最常用）。

遍历方式	访问顺序	应用
先序遍历	根 → 左 → 右	输出树结构
中序遍历	左 → 根 → 右	BST 有序输出
后序遍历	左 → 右 → 根	计算表达式
层序遍历	逐层从左到右	求树高、宽度

三、二叉树的遍历

四、二叉树的非递归遍历（扩展提高）

利用栈可以模拟递归过程，实现二叉树的非递归遍历。

前序遍历非递归算法：

- 根指针 *bt* 非空时输出结点值并压栈，然后遍历左子树
- *bt* 为空时出栈，得到栈顶结点，然后遍历其右子树
- 循环直到 *bt* 为空且栈为空

中序遍历非递归算法：

- 与前序遍历类似，区别在于：出栈后再输出结点值
- 即：左子树遍历完毕后，弹出栈顶结点并访问，然后遍历右子树

后序遍历非递归算法：

- 每个结点需入栈两次、出栈两次
- 设置标志 *flag*：1 表示第一次出栈（左子树遍历完毕，不能访问），2 表示第二次出栈（右子树遍历完毕，可以访问）
- 第一次出栈时修改标志为 2，继续遍历右子树
- 第二次出栈时输出结点值

五、树、森林与二叉树的转换

- 树 → 二叉树：孩子兄弟表示法（左孩子，右兄弟）
- 森林 → 二叉树：树根视为兄弟，依次连接

本章习题

（一）选择题

1. 一棵深度为 k 的二叉树最多有 () 个结点。
A. 2^{k-1}
A. $2^k - 1$
A. $2^{k+1} - 1$
A. 2^k
2. 某二叉树的中序遍历序列为 DBEAF C, 先序遍历序列为 ABDECF, 则后序遍历序列为 ()。
A. DEBFCA
A. DEBFC A
A. DEFBCA
A. D E B A C F
3. 在二叉树中, 度为 0 的结点数为 n_0 , 度为 2 的结点数为 n_2 , 则 ()。
A. $n_0 = n_2 + 1$
A. $n_0 = n_2$
A. $n_0 = n_2 - 1$
A. $n_0 = 2n_2$
4. 完全二叉树有 1000 个结点, 则该二叉树中叶子结点数为 ()。
A. 500
A. 501
A. 499
A. 250
5. 二叉树的先序遍历相当于 ()。
A. 图的深度优先搜索
A. 图的广度优先搜索
A. 线性表的遍历

- A. 树的层序遍历
6. 树最适合用来表示 ()。
- A. 有序数据
- A. 无序数据
- A. 层次关系的数据
- A. 元素间无关系的数据
7. 将一棵树转换为二叉树后，根结点的 ()。
- A. 左子树为空
- A. 右子树为空
- A. 左右子树均非空
- A. 左右子树均为空
8. 以下关于二叉树的说法，正确的是 ()。
- A. 二叉树是度为 2 的树
- A. 二叉树中每个结点最多有两个孩子
- A. 二叉树中结点的度可以是 3
- A. 二叉树和普通树没有区别
9. 一棵满二叉树有 $2^k - 1$ 个结点，则叶子结点数为 ()。
- A. 2^{k-1}
- A. 2^k
- A. $2^k + 1$
- A. $2^{k-1} + 1$
10. 二叉树的中序遍历 ()。
- A. 可以唯一确定一棵二叉树
- A. 需要配合先序或后序才能唯一确定
- A. 不能用于确定二叉树
- A. 与先序遍历等价

（二）辨析判断题

11. 二叉树的度可能为 1。
12. 满二叉树一定是完全二叉树。
13. 二叉树中，叶子结点的个数等于度为 2 的结点数加 1。
14. 先序遍历和中序遍历序列可以唯一确定一棵二叉树。
15. 树的结点至少有一个分支。
16. 二叉树中的每个结点都有左右两个孩子。
17. 完全二叉树可以用顺序存储结构高效存储。
18. 树的深度是指叶子结点的个数。
19. 森林是互不相交的树的集合。
20. 后序遍历表达式树可以得到逆波兰式。

（三）简答题

21. 简述二叉树的性质—— $n_0 = n_2 + 1$ 的推导思路。
22. 给出先序、中序和后序遍历的定义，并说明中序与先序如何配合唯一确定二叉树。
23. 将树转换为二叉树的规则是什么？

(四) 编程题

24. 编写 C++ 函数，实现二叉树的先序遍历（递归版本）。
25. 编写 C++ 函数，统计二叉树中叶子结点的个数。
26. 编写 C++ 函数，求二叉树的深度（高度）。
27. 编写 C++ 函数，用非递归算法（栈实现）实现二叉树的中序遍历。

第6章 图

本章知识点总结

一、图的逻辑结构

定义：图 $G = (V, E)$ ， V 是顶点非空有限集， E 是边有限集。

分类：无向图、有向图、完全图、带权图（网）。

术语：子图、度（入度/出度）、连通、连通分量、强连通、生成树（ $n - 1$ 条边）。

二、图的存储结构

邻接矩阵：

- 顶点表 + $n \times n$ 矩阵
- 无向图对称；行和为出度，列和为入度
- 空间复杂度 $O(n^2)$

邻接表：

- 顶点表（数组）+ 边表（链表）
- 空间复杂度 $O(n + e)$
- 有向图给出度，逆邻接表给出入度

三、图的遍历

DFS（深度优先搜索）：递归访问邻接顶点，类似先序遍历。 $O(n^2)$ （邻接矩阵）/ $O(n + e)$ （邻接表）。

BFS（广度优先搜索）：逐层访问邻接顶点，需队列辅助，类似层序遍历。 $O(n^2)$ （邻接矩阵）/ $O(n + e)$ （邻接表）。

四、最小生成树

Prim 算法：每次选与已选顶点集相连的最小权值边，适合边稠密图， $O(n^2)$ 。

Kruskal 算法：边按权值排序，依次选不构成回路的边，适合边稀疏图， $O(e \cdot \log e)$ 。

五、最短路径

Dijkstra 算法（单源点）：

1. 初始化： $S = \{\text{源点}\}$ ，dist 存当前最短路径
2. 每次选 $V-S$ 中 dist 最小的顶点 k 加入 S
3. 更新与 k 相邻顶点的 dist 值
4. 重复直到所有顶点在 S 中

时间复杂度 $O(n^2)$ ，仅适用于非负权值图。

六、拓扑排序

- 适用于有向无环图（DAG）
- 算法：不断删除入度为 0 的顶点
- 应用：课程编排、工程流程规划

本章习题

（一）选择题

1. 具有 n 个顶点的无向完全图有（ ）条边。

A. $n(n-1)/2$

A. $n(n-1)$

A. n^2

A. n

2. 图的深度优先搜索遍历类似于树的（ ）。

A. 先序遍历

A. 中序遍历

A. 后序遍历

A. 层序遍历

3. 邻接表的空间复杂度是（ ）。

A. $O(n)$

A. $O(e)$

- A. $O(n + e)$
 - A. $O(n^2)$
4. 以下算法中适用于求无向连通网最小生成树的是 ()。
- A. Dijkstra 算法
 - A. Prim 算法
 - A. Floyd 算法
 - A. DFS
5. 连通图的生成树包含 () 个顶点和 () 条边。
- A. $n, n - 1$
 - A. n, n
 - A. $n - 1, n$
 - A. $n, n + 1$
6. Dijkstra 算法求最短路径时, 每次从 $V - S$ 中选中的顶点是 ()。
- A. dist 值最小的顶点
 - A. dist 值最大的顶点
 - A. 编号最小的顶点
 - A. 入度最大的顶点
7. 有 n 个顶点 e 条边的有向图, 用邻接表存储, 求所有顶点入度的时间复杂度为 ()。
- A. $O(n)$
 - A. $O(e)$
 - A. $O(n + e)$
 - A. $O(n^2)$
8. 拓扑排序的输出序列中, ()。
- A. 必须包含所有顶点
 - A. 只包含入度为 0 的顶点
 - A. 只包含出度为 0 的顶点
 - A. 可以包含部分顶点
9. 以下关于 BFS 的说法, 正确的是 ()。

- A. 需要用栈辅助
- A. 需要用队列辅助
- A. 需要使用递归
- A. 不需要辅助结构

10. 以下关于图的说法，错误的是（ ）。

- A. 图可以是有向的
- A. 图的顶点集合可以为空
- A. 图的边集合可以为空
- A. 图可以带权

（二）辨析判断题

- 11. 图的邻接矩阵是对称的。
- 12. 连通图的生成树是不唯一的。
- 13. DFS 可以判断有向图中是否存在回路。
- 14. Prim 算法适合边稀疏的图。
- 15. Dijkstra 算法可以处理带负权边的图。
- 16. 拓扑排序只能用于有向无环图。
- 17. 无向图的邻接矩阵一定是对称矩阵。
- 18. 图的深度优先搜索不需要辅助空间。
- 19. BFS 可以用来判断图的连通性。
- 20. 最小生成树的代价在所有生成树中最小。

（三）简答题

- 21. 简述 DFS 和 BFS 的区别。

22. 比较 Prim 算法和 Kruskal 算法的适用场景。

23. 简述 Dijkstra 算法的基本思想和步骤。

（四）编程题

24. 用 C++ 实现图的深度优先搜索（DFS）的递归函数，图以邻接矩阵存储。

25. 编写 C++ 函数，基于邻接矩阵实现图的广度优先搜索（BFS），需使用队列。

26. 编写 C++ 函数，基于邻接矩阵实现 Prim 算法的核心步骤（计算最小生成树的代价）。

第7章 查找技术

本章知识点总结

一、查找的基本概念

关键码：标识记录的数据项（主关键码唯一，次关键码不唯一）。

平均查找长度（ASL）：衡量查找算法效率。

分类：静态查找（仅查找）和动态查找（查找中插入/删除）。

二、顺序查找与折半查找

顺序查找： $O(n)$ ， $ASL_{成功} = (n + 1)/2$ ， $ASL_{失败} = n + 1$ 。适用于顺序表和链表，无需有序。

折半查找：仅适用于有序顺序表。 $O(\log_2 n)$ 。每次取中间元素比较，缩小一半范围。

三、二叉排序树（BST）

定义：左子树所有结点值 < 根值 < 右子树所有结点值，且左右子树均为 BST。

性质：中序遍历得升序序列。

查找：与根比较，递归进入左或右子树。

插入：查找失败的位置即为插入位置（新结点总是叶子）。

删除：三种情况处理。

性能：最好 $O(\log_2 n)$ ，最坏 $O(n)$ 。

四、散列查找

基本思想： $addr = H(key)$ ，在关键码和存储地址之间建立映射。

散列函数：

- 直接定址法： $H(key) = a \times key + b$
- 除留余数法： $H(key) = key \% p$ （ p 为质数）
- 数字分析法、平方取中法

冲突处理：

- 开放定址法（线性探测、二次探测）

- 链地址法（同义词放在一个单链表中）

本章习题

（一）选择题

1. 顺序查找的平均查找长度是（ ）。

A. n
A. $(n+1)/2$
A. $\log_2 n$
A. $(n-1)/2$

2. 折半查找的时间复杂度为（ ）。

A. $O(n)$
A. $O(\log_2 n)$
A. $O(n^2)$
A. $O(n \log_2 n)$

3. 以下查找方法中，（ ）要求查找表必须是有序的顺序表。

A. 顺序查找
A. 折半查找
A. 二叉排序树查找
A. 散列查找

4. 二叉排序树中序遍历得到的序列是（ ）。

A. 递减序列
A. 递增序列
A. 无序序列
A. 随机序列

5. 散列表的装填因子 α 反映的是（ ）。

A. 散列表的大小
A. 表中记录数的多少
A. 表的装满程度

- A. 冲突的严重程度
6. 散列查找中，产生冲突的根本原因是（ ）。
- A. 存储空间不足
- A. 散列函数不完美
- A. 数据量太大
- A. 关键码重复
7. 对 n 个元素进行折半查找时，查找成功的最多比较次数为（ ）。
- A. n
- A. $\log_2(n+1)$
- A. $\lfloor \log_2 n \rfloor + 1$
- A. $n/2$
8. 在长度为 10 的有序表中进行折半查找， $ASL_{\text{成功}}$ 约为（ ）。
- A. 2.5
- A. 2.9
- A. 3.3
- A. 3.9
9. 以下不是常见的散列函数构造方法的是（ ）。
- A. 直接定址法
- A. 除留余数法
- A. 折半法
- A. 平方取中法
10. 以下关于 BST 查找的说法，正确的是（ ）。
- A. BST 的查找效率一定比顺序查找高
- A. BST 查找的最坏情况是 $O(n)$
- A. BST 查找的最坏情况是 $O(\log_2 n)$
- A. BST 查找只能使用递归实现

（二）辨析判断题

11. 顺序查找可以用于链表。
12. 折半查找适用于所有存储结构。
13. 二叉排序树的中序遍历结果是递增序列。
14. 散列查找不需要比较关键码。
15. 装填因子越大，冲突的可能性越大。
16. 开放定址法容易出现”堆积”现象。
17. BST 的查找效率与树的形态无关。
18. 除留余数法选择 p 为质数可以降低冲突。
19. 链地址法将同义词存放在同一链表中。
20. 折半查找的平均查找长度一定小于顺序查找。

（三）简答题

21. 简述折半查找的适用条件和基本思想。
22. 什么是冲突？哈希表中处理冲突的常用方法有哪些？
23. 二叉排序树的查找效率受什么因素影响？如何优化？

(四) 编程题

24. 用 C++ 实现折半查找算法（非递归版本）。
25. 编写 C++ 函数，在二叉排序树中插入一个结点。
26. 编写 C++ 函数，实现除留余数法的散列函数，并用链地址法处理冲突（只需插入操作）。

第8章 排序技术

本章知识点总结

一、排序的基本概念

排序：将记录按关键码递增或递减顺序排列。

稳定性：相等关键码的相对次序不变，则排序稳定。

内排序与外排序：内排序在内存中完成，外排序需使用外存。

二、插入排序

直接插入排序：依次将每个元素插入到已排序的序列中。

- 最好 $O(n)$ ，最坏 $O(n^2)$ ，平均 $O(n^2)$
- 稳定排序

三、交换排序

冒泡排序：相邻元素比较，大者后移。

- 最好 $O(n)$ （已有序），最坏 $O(n^2)$ ，平均 $O(n^2)$
- 稳定排序

快速排序：选基准，分区，递归排序。

- 最好 $O(n \log_2 n)$ ，最坏 $O(n^2)$ ，平均 $O(n \log_2 n)$
- 不稳定排序

四、选择排序

简单选择排序：每次选最小元素放到已排序末尾。

- 始终 $O(n^2)$
- 不稳定排序

堆排序：利用大根堆，每次取堆顶最大元素。

- 始终 $O(n \log_2 n)$
- 不稳定排序

五、归并排序

二路归并排序：两两合并有序序列。

- 始终 $O(n \log_2 n)$
- 稳定排序
- 需要 $O(n)$ 辅助空间

六、基数排序

- 基于关键码的各位进行分配和收集
- 时间复杂度 $O(d(n+r))$ ， d 为位数， r 为基数
- 稳定排序

各种排序算法比较

排序算法	最好	平均	最坏	稳定性
直接插入	$O(n)$	$O(n^2)$	$O(n^2)$	稳定
冒泡排序	$O(n)$	$O(n^2)$	$O(n^2)$	稳定
快速排序	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	不稳定
简单选择	$O(n^2)$	$O(n^2)$	$O(n^2)$	不稳定
堆排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	不稳定
归并排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	稳定
基数排序	$O(d(n+r))$	$O(d(n+r))$	$O(d(n+r))$	稳定

本章习题

（一）选择题

1. 在待排序元素基本有序的情况下，运行效率最高的排序算法是（ ）。
A. 直接插入排序
A. 快速排序
A. 简单选择排序
A. 堆排序
2. 以下排序算法中，不稳定的是（ ）。

- A. 直接插入排序
 - A. 冒泡排序
 - A. 归并排序
 - A. 快速排序
3. 快速排序的最坏时间复杂度为 ()。
- A. $O(n)$
 - A. $O(n \log_2 n)$
 - A. $O(n^2)$
 - A. $O(\log_2 n)$
4. 堆排序的时间复杂度为 ()。
- A. $O(n)$
 - A. $O(n \log_2 n)$
 - A. $O(n^2)$
 - A. $O(\log_2 n)$
5. 归并排序需要的辅助空间为 ()。
- A. $O(1)$
 - A. $O(\log_2 n)$
 - A. $O(n)$
 - A. $O(n^2)$
6. 以下排序算法中, 平均时间复杂度为 $O(n \log_2 n)$ 的是 ()。
- A. 冒泡排序
 - A. 直接插入排序
 - A. 简单选择排序
 - A. 快速排序
7. 稳定排序是指 ()。
- A. 排序后数据量不变
 - A. 排序后相等关键码的相对次序不变
 - A. 排序后序列有序

A. 排序算法的时间复杂度固定

8. 冒泡排序在最好情况下的时间复杂度为 ()。

A. $O(n)$

A. $O(n \log_2 n)$

A. $O(n^2)$

A. $O(1)$

9. 以下排序算法中，需要最大辅助空间的是 ()。

A. 堆排序

A. 快速排序

A. 归并排序

A. 简单选择排序

10. 对 n 个不同的关键字进行冒泡排序，最大的比较次数为 ()。

A. $n - 1$

A. $n(n - 1)/2$

A. n^2

A. n

(二) 辨析判断题

11. 直接插入排序是稳定的排序算法。

12. 快速排序在任何情况下都是 $O(n \log_2 n)$ 。

13. 堆排序适用于记录数较多的场景。

14. 归并排序需要 $O(n)$ 的辅助空间。

15. 基数排序是基于关键码比较的排序算法。

16. 冒泡排序在数据完全有序时只需 $O(n)$ 时间。

17. 简单选择排序在任何情况下的比较次数都是 $n(n - 1)/2$ 。

18. 快速排序是一种不稳定的排序算法。

19. 堆排序适用于数据量较小的场景。

20. 外排序适用于数据量大、无法一次装入内存的场景。

（三）简答题

21. 什么是排序算法的稳定性？举例说明。

22. 简述快速排序的基本思想。

23. 比较直接插入排序、冒泡排序和简单选择排序的异同。

（四）编程题

24. 用 C++ 实现冒泡排序算法。

25. 用 C++ 实现快速排序算法（递归版本）。

26. 用 C++ 实现直接插入排序算法。

第 A 章 习题答案与解析

第 1 章绪论答案

选择题

1. **B**。数据结构在内存中的表示是存储结构，逻辑结构是用户视图。
2. **C**。算法的五大特性是：有穷性、确定性、可行性、输入、输出。可读性是好算法的要求，但不是算法的基本特性。
3. **B**。对数阶 $O(\log_2 n)$ 增长最慢。
4. **C**。ADT 抽象掉的就是实现细节。
5. **B**。线性结构数据元素之间是一一对应的关系。
6. **C**。伪代码表达能力强、易于理解、易于实现，推荐用于描述算法。
7. **A**。时间复杂度是问题规模 n 的函数，与计算机硬件和语言环境无关。
8. **D**。好算法应当正确、健壮、可理解、高效，并非越复杂越好。
9. **C**。算法分析的目的在于评价和指导改进算法效率。
10. **B**。顺序表中逻辑关系由存储位置（相邻关系）表示。

判断题

11. $\checkmark$ 。自然语言、流程图、伪代码、程序语言都是描述算法的常用方法。
12. $\times$ 。逻辑结构是用户视图，面向问题；存储结构才是数据在内存中的表示。
13. $\checkmark$ 。ADT 只关心逻辑结构和操作接口，不关心具体实现。
14. $\times$ 。算法可以有零个输入。
15. $\times$ 。 $O(2^n)$ 是指数时间，属于难解问题。多项式时间指 $O(n^k)$ 。
16. $\checkmark$ 。线性结构可以用顺序或链接两种方式存储。
17. $\times$ 。健壮性是指对非法输入的抵抗能力；正确性才是指能正确解决问题。

18. ✓。7±2 原则是米勒原则，强调合适的抽象分级。
19. ✓。存储结构实质上是如何分配内存来表示数据及其关系。
20. ✓。递归算法通过建立递推关系式分析时间复杂度。

简答题

21. 逻辑结构是用户视图，面向问题本身，描述数据元素之间的逻辑关系；存储结构是实现视图，面向计算机，描述数据在内存中的存储方式。同一逻辑结构可采用多种存储结构实现，不同存储结构影响数据处理的效率。
22. 有穷性指算法在执行有穷步后必须结束，且每一步在有穷时间内完成。例如：“质因数分解求最大公约数”算法中，如果尝试用质因数分解法（目前尚未找到多项式时间算法），对于大整数可能在实际可接受时间内无法完成，不满足有穷性。另一个例子：死循环（如 while(true) 没有 break 条件）不满足有穷性。
23. 大 O 记号：若存在正的常数 c 和 n_0 ，使得对所有 $n \geq n_0$ 都有 $T(n) \leq c \cdot f(n)$ ，则记 $T(n) = O(f(n))$ 。它表示算法运行时间的增长率上限，用于在不同数量级上比较算法效率，忽略低次项和系数。

编程题

```
24.
1  int Sum(int A[], int n) {
2      int sum = 0;
3      for (int i = 0; i < n; i++)
4          sum += A[i];
5      return sum;
6  }
7  // 时间复杂度: O(n)
```

```
1  const int MaxSize = 100;
2  class SeqList {
3  public:
4      int data[MaxSize];
5      int length;
6      int Locate(int x) {
7          for (int i = 0; i < length; i++)
8              if (data[i] == x) return i;
9          return -1; // 未找到
10     }
11 };
```

```
26.
1 struct Node { int data; Node* next; };
2 int Count(Node* head) {
3     int cnt = 0;
4     Node* p = head->next; // 跳过头结点
5     while (p != NULL) {
6         cnt++;
7         p = p->next;
8     }
9     return cnt;
10 }
```

第 2 章线性表

选择题

1. A。顺序存储要求存储单元连续。
2. A。先在 s 的 $next$ 中保存 p 的原后继，再将 p 的 $next$ 指向 s 。
3. B。平均移动 $n/2$ 个元素。
4. D。链式存储的结点可以在内存任意位置，连续与否均可。
5. D。带尾指针的循环单链表，在表尾插入 $O(1)$ ，且可通过尾指针的 $next$ 访问头结点。
6. A。顺序表地址是下标的线性函数： $Loc(a_i) = Loc(a_0) + i \times c$ 。
7. C。最后应执行 p 前驱的 $next$ 指向 s 。
8. D。带尾指针的单循环链表，在末尾后插入 $O(1)$ ，删除第一个元素 $O(1)$ 。
9. B。顺序表支持随机存取， $O(1)$ 。
10. B。链表是顺序存取结构。

判断题

11. ×。第一个元素无前驱，最后一个元素无后继。
12. √。顺序表可通过下标直接访问任一元素。
13. √。链表删除只需修改指针。
14. √。循环链表尾指针指向头结点（或首结点），没有 NULL。

15. ✓。顺序表只需存储数据，链表还需存储指针。
16. ×。已知前驱位置时插入为 $O(1)$ ，若需先查找则为 $O(n)$ 。
17. ✓。双向链表多一个 prior 指针域。
18. ×。线性表长度动态变化（尤其是链表）。
19. ✓。顺序表可通过下标随机访问。
20. ✓。链表插入删除只需修改指针，适合频繁操作。

简答题

21. 顺序表优点：随机存取、存储密度高；缺点：插入删除需移动元素、需预分配空间。链表优点：动态存储、插入删除只需修改指针；缺点：顺序存取、存储密度低。顺序表适用于频繁查找的场景，链表适用于频繁插入删除的场景。
22. 头结点是在单链表第一个元素结点之前附加的一个结点，其数据域不存储任何有效数据。好处：对第一个元素的操作与对其他元素一致，无需特殊处理；空表和非空表的处理统一。
23. 循环链表尾结点指向头结点，可从任一点遍历全表；双向链表每个结点有 prior 和 next 两个指针，可双向遍历。循环链表节省一个方向的指针（相比双向），双向链表可方便地寻找前驱。

编程题

```
24.
1 void DeleteX(Node* head, int x) {
2     Node* p = head;
3     while (p->next != NULL) {
4         if (p->next->data == x) {
5             Node* q = p->next;
6             p->next = q->next;
7             delete q;
8         } else {
9             p = p->next;
10        }
11    }
12 }
```

```
1 bool Insert(SeqList &L, int i, int e) {  
2     if (i < 0 || i > L.length || L.length >= MaxSize)  
3         return false;  
4     for (int j = L.length - 1; j >= i; j--)  
5         L.data[j+1] = L.data[j];  
6     L.data[i] = e;  
7     L.length++;  
8     return true;  
9 }
```

26.

```
1 Node* Merge(Node* ha, Node* hb) {  
2     Node* hc = new Node;  
3     Node* pc = hc;  
4     Node* pa = ha->next;  
5     Node* pb = hb->next;  
6     while (pa != NULL && pb != NULL) {  
7         if (pa->data <= pb->data) {  
8             pc->next = pa; pa = pa->next;  
9         } else {  
10            pc->next = pb; pb = pb->next;  
11        }  
12        pc = pc->next;  
13    }  
14    pc->next = (pa != NULL) ? pa : pb;  
15    return hc;  
16 }
```

第 3 章栈和队列

选择题

1. C。栈和队列都只允许在端点进行插入和删除。
2. D。3 出栈时 1 和 2 还在栈中，1 在栈底，不可能在 2 之前出栈。
3. A。循环队列判空条件为 $\text{front} = \text{rear}$ 。
4. C。任务调度是队列的典型应用，而不是栈。
5. B。链栈动态分配，不需要判满。

6. **C**。循环队列元素个数计算公式。
7. **D**。a 先入栈，b 再入栈，然后 d 入栈出栈，e 入栈出栈，c 出栈，此时 a 和 b 在栈中（b 在上），不可能 a 先出栈。
8. **C**。FIFO：先进先出。
9. **A**。5 个不同元素入栈的出栈序列数为 Catalan 数 $C_5 = \frac{1}{5+1} \binom{10}{5} = 42$ 。
10. **B**。链栈入栈用头插法，即在栈顶插入。

判断题

11. $\checkmark$ 。栈是操作受限的线性表。
12. $\checkmark$ 。栈和队列都是线性结构。
13. $\times$ 。front = rear 是队空条件，不是队满。
14. $\checkmark$ 。左括号入栈，右括号与栈顶匹配。
15. $\times$ 。链栈动态分配，不需要判满。
16. $\checkmark$ 。队列只允许在队尾插入。
17. $\checkmark$ 。循环队列通过取模运算复用已出队的空间。
18. $\checkmark$ 。顺序栈用数组存储，判满条件为 top = StackSize-1。
19. $\checkmark$ 。出队前需判空。
20. $\checkmark$ 。两栈共享空间可以更灵活地分配空间。

简答题

21. 栈和队列都是操作受限的线性表。栈只允许在一端（栈顶）操作，遵循后进先出（LIFO）；队列允许在一端（队尾）插入、另一端（队头）删除，遵循先进先出（FIFO）。
22. “假溢出”是指顺序队列中，rear 已到数组末尾而 front 前面还有空闲空间，但无法再入队的现象。循环队列通过将数组视为首尾相接的环形结构，使用取模运算 $\text{rear} = (\text{rear} + 1) \% m$ 来实现空间复用，解决了假溢出问题。
23. 表达式求值（如 $3 + 4 \times 2$ ）中，使用两个栈：操作数栈和运算符栈。扫描表达式，遇到操作数入操作数栈，遇到运算符则与运算符栈顶比较优先级，优先级高则入栈，优先级低则弹出栈顶运算符并计算。最终操作数栈顶即为结果。

编程题

```
24.
1  const int StackSize = 100;
2  template <class DataType>
3  class SeqStack {
4  private:
5      DataType data[StackSize];
6      int top;
7  public:
8      SeqStack() { top = -1; }
9      void Push(DataType x) {
10         if (top == StackSize - 1) throw "overflow";
11         data[++top] = x;
12     }
13     DataType Pop() {
14         if (top == -1) throw "underflow";
15         return data[top--];
16     }
17 };
```

```
1  bool Match(char exp[]) {
2      SeqStack<char> s;
3      for (int i = 0; exp[i] != '\0'; i++) {
4          if (exp[i] == '(') s.Push('(');
5          else if (exp[i] == ')') {
6              if (s.Empty()) return false;
7              s.Pop();
8          }
9      }
10     return s.Empty();
11 }
```

```
26.
1  struct Node { int data; Node* next; };
2  class LinkQueue {
3  private:
4      Node *front, *rear;
5  public:
6      LinkQueue() {
7          front = rear = new Node;
8          front->next = NULL;
9      }
```

```
10     void EnQueue(int x) {
11         Node* s = new Node{x, NULL};
12         rear->next = s;
13         rear = s;
14     }
15     int DeQueue() {
16         if (front == rear) throw "empty";
17         Node* p = front->next;
18         int x = p->data;
19         front->next = p->next;
20         if (p == rear) rear = front;
21         delete p;
22         return x;
23     }
24 };
```

第 4 章字符串和多维数组

选择题

1. A。字符串是有限个字符的序列。
2. B。BF 最坏情况为 $O(n \times m)$ 。
3. C。next 数组记录模式串的最长相等前后缀长度。
4. A。按行优先：地址 = 基址 + $((i - 1) \times n + (j - 1)) \times 4$ 。
5. B。特殊矩阵压缩存储为值相同的元素只分配一个空间。
6. B。 n 阶对称矩阵只需 $n(n + 1)/2$ 个存储单元。
7. C。按列优先， $A[i][j]$ 前面有 $j - 1$ 列。
8. A。空串长度为 0。
9. C。KMP 最大优势是主串不需要回溯。
10. C。数组是线性表的推广。

判断题

11. ×。KMP 时间复杂度为 $O(n + m)$ 。

12. ✓。空串长度为零。
13. ✓。对称矩阵只需存储上三角或下三角。
14. ✓。数组是线性结构。
15. ✓。数组不支持插入和删除。
16. ✓。BF 匹配失败时主串指针需回溯。
17. ✓。next 数组只与模式串本身有关。
18. ×。压缩存储后仍支持随机存取。
19. ✓。一维数组是线性表。
20. ×。三角矩阵和对角矩阵存储的单元数不同。

简答题

21. BF 算法简单直观但效率较低，主串需要回溯，时间复杂度 $O(n \times m)$ 。KMP 算法利用 next 数组记录已匹配信息，主串无需回溯，时间复杂度 $O(n + m)$ 。KMP 是 BF 的改进。
22. 对称矩阵中 $a_{ij} = a_{ji}$ ，只需存储下三角（或上三角）部分。 n 阶对称矩阵下三角元素 a_{ij} ($i \geq j$) 在一维数组中的下标为 $k = i(i-1)/2 + j - 1$ (C++ 下标从 0 开始)。
23. 按行优先：先存第一行所有列，再存第二行所有列……对 n 维数组，依次累加各维的偏移量。如三维数组 $A[m_1][m_2][m_3]$ ， $A[i][j][k]$ 的地址为 $\text{Loc}(a_{000}) + (i \times m_2 \times m_3 + j \times m_3 + k) \times c$ 。

编程题

```

24.
1  int BF(char S[], char T[]) {
2      int i = 0, j = 0;
3      while (S[i] != '\0' && T[j] != '\0') {
4          if (S[i] == T[j]) { i++; j++; }
5          else { i = i - j + 1; j = 0; }
6      }
7      if (T[j] == '\0') return i - j;
8      return -1;
9  }

```

```

1  int SymIndex(int i, int j) {
2      // 对称矩阵下三角 a_ij (i>=j) 的一维数组下标
3      if (i < j) { int t = i; i = j; j = t; }

```



```
4     return i * (i - 1) / 2 + j - 1;  
5 }
```

```
26.  
1 int Loc3D(int base, int m1, int m2, int m3,  
2         int i, int j, int k, int size) {  
3     return base + (i * m2 * m3 + j * m3 + k) * size;  
4 }
```

第 5 章树和二叉树

选择题

1. B。深度 k 的二叉树最多 $2^k - 1$ 个结点。
2. A。由先序 ABDECF 和中序 DBEAF C 可重构二叉树，后序为 DEBFCA。
3. A。二叉树性质： $n_0 = n_2 + 1$ 。
4. A。完全二叉树中叶子结点数为 $\lceil n/2 \rceil = 500$ 。
5. A。DFS 类似于树的先序遍历。
6. C。树适合表示层次关系。
7. B。树转换为二叉树后，根结点无兄弟，右子树为空。
8. B。每个结点最多有两个孩子。
9. A。满二叉树第 k 层叶子数为 2^{k-1} 。
10. B。中序需配合先序或后序才能唯一确定二叉树。

判断题

11. ✓。二叉树中结点度可为 0、1、2。
12. ✓。满二叉树是完全二叉树的特例。
13. ✓。 $n_0 = n_2 + 1$ 。
14. ✓。先序 + 中序可唯一确定二叉树。
15. ×。叶子结点没有分支。
16. ×。二叉树结点可以有 0、1 或 2 个孩子。

17. $\checkmark$ 。完全二叉树可用数组顺序存储。
18. $\times$ 。树的深度是结点的最大层次数。
19. $\checkmark$ 。森林是 m 棵互不相交的树的集合。
20. $\checkmark$ 。后序遍历表达式树得逆波兰式。

简答题

21. 设二叉树中度为 0、1、2 的结点数分别为 n_0 、 n_1 、 n_2 ，总结点数 $n = n_0 + n_1 + n_2$ 。从边的角度看，除根结点外每个结点有一条入边，总边数 $= n - 1 = n_1 + 2n_2$ 。两式联立消去 n_1 得 $n_0 = n_2 + 1$ 。
22. 先序（根左右）、中序（左根右）、后序（左右根）。中序配合先序可唯一确定二叉树：先序第一个元素为根，在中序中找到根，左边为左子树、右边为右子树，递归构造。
23. 规则：左孩子右兄弟。将根结点的第一个孩子作为左孩子，将兄弟结点依次链接为右孩子。这样，任何一棵树转换为二叉树后，根结点没有右子树。

编程题

24.

```

1 struct Node { int data; Node *lchild, *rchild; };
2 void PreOrder(Node* t) {
3     if (t == NULL) return;
4     cout << t->data << "□";
5     PreOrder(t->lchild);
6     PreOrder(t->rchild);
7 }
```

```

1 int CountLeaf(Node* t) {
2     if (t == NULL) return 0;
3     if (t->lchild == NULL && t->rchild == NULL)
4         return 1;
5     return CountLeaf(t->lchild) + CountLeaf(t->rchild);
6 }
```

26.

```

1 int Depth(Node* t) {
2     if (t == NULL) return 0;
3     int l = Depth(t->lchild);
4     int r = Depth(t->rchild);
5     return (l > r ? l : r) + 1;
6 }
```

```
1 void InOrderNonRec(Node* t) {  
2     Node* s[100]; int top = -1;  
3     Node* p = t;  
4     while (p != NULL || top != -1) {  
5         while (p != NULL) {  
6             s[++top] = p;  
7             p = p->lchild;  
8         }  
9         if (top != -1) {  
10            p = s[top--];  
11            cout << p->data << "□";  
12            p = p->rchild;  
13        }  
14    }  
15 }
```

第 6 章图

选择题

27. **A**。 n 个顶点的无向完全图有 $n(n-1)/2$ 条边。
2. **A**。 DFS 类似于树的先序遍历。
3. **C**。 邻接表空间复杂度 $O(n+e)$ 。
4. **B**。 Prim 和 Kruskal 是求最小生成树的算法。
5. **A**。 连通图生成树有 n 个顶点、 $n-1$ 条边。
6. **A**。 Dijkstra 每步选 dist 最小的顶点。
7. **C**。 求所有顶点入度需遍历所有边表 $O(n+e)$ 。
8. **A**。 拓扑排序输出序列包含所有顶点。
9. **B**。 BFS 需用队列辅助。
10. **B**。 图的顶点集合不能为空。

判断题

11. ×。无向图的邻接矩阵对称，有向图不一定。
12. ✓。连通图可能有多个生成树。
13. ✓。DFS 的递归过程中若遇到已访问顶点，说明有回路。
14. ×。Prim 适合边稠密图，Kruskal 适合边稀疏图。
15. ×。Dijkstra 不能处理负权边。
16. ✓。有环图无法进行拓扑排序。
17. ✓。无向图邻接矩阵必对称。
18. ×。DFS 需 visited 数组和递归栈空间。
19. ✓。从一个顶点出发 BFS 可访问所有顶点即为连通。
20. ✓。最小生成树是代价最小的生成树。

简答题

21. DFS 用递归（或栈）实现，沿着一条路径深入，遇到死胡同则回溯，类似树的先序遍历。BFS 用队列实现，逐层向外扩展，类似树的层序遍历。DFS 适合寻找路径和回路，BFS 适合求最短路径（无权图）和判断连通性。
22. Prim 算法从顶点出发，每次选择与已选顶点集相连的最小权值边，时间复杂度 $O(n^2)$ ，适合边稠密图。Kruskal 算法从边出发，将所有边按权值排序后依次选取不构成回路的边，时间复杂度 $O(e \log e)$ ，适合边稀疏图。
23. Dijkstra 算法步骤： 初始化 S 含源点，dist 数组存源点到各顶点的当前最短距离； 从 $V - S$ 中选 dist 最小的顶点 k 加入 S ； 更新与 k 相邻顶点的 dist 值； 重复 直到所有顶点在 S 中。

编程题

```
24.
1  const int Max = 100;
2  int visited[Max];
3  void DFS(int G[][Max], int v, int n) {
4      cout << v << "┐";
5      visited[v] = 1;
6      for (int i = 0; i < n; i++)
7          if (G[v][i] && !visited[i])
```

```
8         DFS(G, i, n);
9     }
```

```
1 void BFS(int G[][Max], int v, int n) {
2     queue<int> q;
3     visited[v] = 1;
4     q.push(v);
5     while (!q.empty()) {
6         int u = q.front(); q.pop();
7         cout << u << "□";
8         for (int i = 0; i < n; i++)
9             if (G[u][i] && !visited[i]) {
10                 visited[i] = 1;
11                 q.push(i);
12             }
13     }
14 }
```

26.

```
1 const int INF = 0x3f3f3f3f;
2 int Prim(int G[][Max], int n) {
3     int lowcost[Max], sum = 0;
4     for (int i = 0; i < n; i++)
5         lowcost[i] = G[0][i];
6     lowcost[0] = -1;
7     for (int k = 1; k < n; k++) {
8         int minv = INF, idx = -1;
9         for (int i = 0; i < n; i++)
10             if (lowcost[i] != -1 && lowcost[i] < minv)
11                 { minv = lowcost[i]; idx = i; }
12         sum += minv;
13         lowcost[idx] = -1;
14         for (int i = 0; i < n; i++)
15             if (G[idx][i] < lowcost[i])
16                 lowcost[i] = G[idx][i];
17     }
18     return sum;
19 }
```

第 7 章查找技术

选择题

1. B。顺序查找 $ASL_{成功} = (n + 1)/2$ 。
2. B。折半查找 $O(\log_2 n)$ 。
3. B。折半查找要求有序的顺序表。
4. B。BST 中序遍历为递增序列。
5. C。装填因子反映表的装满程度。
6. B。冲突的根本原因是散列函数将不同关键码映射到同一地址。
7. C。折半查找最多比较 $\lfloor \log_2 n \rfloor + 1$ 次。
8. B。10 个元素折半查找的 $ASL_{成功} = (1 \times 1 + 2 \times 2 + 3 \times 4 + 4 \times 3)/10 \approx 2.9$ 。
9. C。折半法是查找方法，不是散列函数构造方法。
10. B。BST 最坏情况（单支树）为 $O(n)$ 。

判断题

11. ✓。顺序查找适用于线性表，包括链表。
12. ×。折半查找要求顺序存储（数组）。
13. ✓。BST 中序遍历得递增序列。
14. ×。散列查找仍需要比较关键码（解决冲突时）。
15. ✓。装填因子越大，冲突概率越大。
16. ✓。线性探测易导致“堆积”。
17. ×。BST 效率与树形态密切相关。
18. ✓。 p 取质数可减少冲突。
19. ✓。链地址法将同义词放入同一链表。
20. ✓。折半查找 ASL 为 $O(\log n)$ ，顺序查找为 $O(n)$ 。

简答题

21. 适用条件：有序的顺序表（数组）。基本思想：每次取中间元素与待查值比较，若相等则查找成功；若待查值小于中间元素，则在左半部分继续查找；否则在右半部分继续查找，直到找到或区间为空。
22. 冲突是指不同关键码通过散列函数映射到同一地址。处理冲突的常用方法：开放定址法（线性探测再散列、二次探测再散列）；链地址法（同义词存入同一单链表）；再散列法（使用多个散列函数）。
23. BST 查找效率受树形态影响：平衡时 $O(\log_2 n)$ ，单支树时退化为 $O(n)$ 。优化方法：使用平衡二叉树（AVL）或红黑树，使树保持平衡。

编程题

```
24.
1  int BinarySearch(int A[], int n, int key) {
2      int low = 0, high = n - 1;
3      while (low <= high) {
4          int mid = (low + high) / 2;
5          if (A[mid] == key) return mid;
6          else if (A[mid] > key) high = mid - 1;
7          else low = mid + 1;
8      }
9      return -1;
10 }
```

```
1  struct Node { int data; Node *lchild, *rchild; };
2  Node* InsertBST(Node* t, int key) {
3      if (t == NULL) {
4          t = new Node{key, NULL, NULL};
5          return t;
6      }
7      if (key < t->data)
8          t->lchild = InsertBST(t->lchild, key);
9      else if (key > t->data)
10         t->rchild = InsertBST(t->rchild, key);
11     return t;
12 }
```

```
26.
1  const int M = 13; // 散列表大小（质数）
2  struct Node { int key; Node* next; };
```

```
3 Node* HT[M]; // 链地址法散列表
4
5 int H(int key) { return key % M; }
6
7 void Insert(int key) {
8     int addr = H(key);
9     Node* s = new Node{key, HT[addr]};
10    HT[addr] = s;
11 }
```

第 8 章排序技术

选择题

1. **A**。基本有序时，直接插入排序只需少量比较， $O(n)$ 。
2. **D**。快速排序是不稳定的。
3. **C**。快速排序最坏情况 $O(n^2)$ 。
4. **B**。堆排序始终 $O(n \log_2 n)$ 。
5. **C**。归并排序需 $O(n)$ 辅助空间。
6. **D**。快速排序平均 $O(n \log_2 n)$ 。
7. **B**。稳定排序指相等关键码的相对次序不变。
8. **A**。冒泡排序最好情况 $O(n)$ （已有序，一次扫描无交换即可结束）。
9. **C**。归并排序需要 $O(n)$ 的辅助空间。
10. **B**。冒泡排序最大比较次数为 $n(n-1)/2$ 。

判断题

11. $\checkmark$ 。直接插入排序是稳定的。
12. $\times$ 。快速排序最坏情况 $O(n^2)$ 。
13. $\checkmark$ 。堆排序适合大数据量。
14. $\checkmark$ 。归并排序需要 $O(n)$ 辅助空间。
15. $\times$ 。基数排序不是基于比较的排序。

- 16. ✓。加标志位的冒泡排序在有序时仅需一轮扫描。
- 17. ✓。简单选择排序比较次数与初始序列无关。
- 18. ✓。快速排序不稳定。
- 19. ×。堆排序更适合大数据量。
- 20. ✓。外排序处理无法全部装入内存的数据。

简答题

- 21. 稳定性是指排序后相等关键码的相对次序不变。例如，对记录 $\{(3, a), (1, b), (3, c)\}$ 按数字排序，稳定排序结果是 $\{(1, b), (3, a), (3, c)\}$ ，不稳定排序可能得到 $\{(1, b), (3, c), (3, a)\}$ 。
- 22. 快速排序基本思想：选一个基准元素（如第一个）；分区（partition），将小于基准的放左边，大于基准的放右边；递归对左右两个子区间进行快速排序。核心是每次确定一个元素的最终位置。
- 23. 三者相同点：时间复杂度平均都是 $O(n^2)$ 。不同点：插入排序在基本有序时最好可达 $O(n)$ ；冒泡排序在有序时最好可达 $O(n)$ ；简单选择排序比较次数固定为 $n(n-1)/2$ ，与初始顺序无关。三者中仅直接插入排序和冒泡排序是稳定的。

编程题

```
24.
1 void BubbleSort(int r[], int n) {
2     for (int i = 1; i < n; i++) {
3         bool flag = false;
4         for (int j = 0; j < n - i; j++)
5             if (r[j] > r[j+1]) {
6                 int t = r[j];
7                 r[j] = r[j+1];
8                 r[j+1] = t;
9                 flag = true;
10            }
11        if (!flag) break;
12    }
13 }
```

```
1 int Partition(int r[], int low, int high) {
2     int pivot = r[low];
3     while (low < high) {
4         while (low < high && r[high] >= pivot) high--;
```

```
5         r[low] = r[high];
6         while (low < high && r[low] <= pivot) low++;
7         r[high] = r[low];
8     }
9     r[low] = pivot;
10    return low;
11 }
12 void QuickSort(int r[], int low, int high) {
13     if (low < high) {
14         int pos = Partition(r, low, high);
15         QuickSort(r, low, pos - 1);
16         QuickSort(r, pos + 1, high);
17     }
18 }
```

25.

```
1 void InsertSort(int r[], int n) {
2     for (int i = 1; i < n; i++) {
3         int temp = r[i];
4         int j = i - 1;
5         while (j >= 0 && r[j] > temp) {
6             r[j + 1] = r[j];
7             j--;
8         }
9         r[j + 1] = temp;
10    }
11 }
```

第 B 章 模拟试题

一、选择题

1. 数据结构在计算机内存中的表示是指 ()。
A. 数据的逻辑结构 B. 数据的存储结构 C. 数据的抽象运算 D. 数据元素的类型
2. 以下关于栈的描述, 正确的是 ()。
A. 栈允许在任意位置插入和删除 B. 栈是一种后进先出的线性表 C. 栈的典型应用是任务调度
D. 栈的删除操作在队尾进行
3. 某二叉树中度为 2 的结点有 10 个, 则叶子结点的个数为 ()。
A. 9 B. 10 C. 11 D. 12
4. 图的广度优先搜索 (BFS) 需要使用 () 来辅助实现。
A. 栈 B. 队列 C. 递归函数 D. 数组
5. 折半查找要求查找表必须是 ()。
A. 顺序存储且有序 B. 链式存储且有序 C. 顺序存储, 有序与否均可 D. 链式存储, 无序即可
6. 以下排序算法中, 在最好情况下时间复杂度仍为 $O(n^2)$ 的是 ()。
A. 直接插入排序 B. 冒泡排序 C. 快速排序 D. 简单选择排序
7. 单链表中, 若要在已知结点 p 之后插入新结点 s , 正确的操作是 ()。
A. $p \rightarrow \text{next} = s; s \rightarrow \text{next} = p$ B. $s \rightarrow \text{next} = p \rightarrow \text{next}; p \rightarrow \text{next} = s$ C. $s \rightarrow \text{next} = p; p \rightarrow \text{next} = s \rightarrow \text{next}$ D. $p \rightarrow \text{next} = s \rightarrow \text{next}; s \rightarrow \text{next} = p$
8. KMP 算法的时间复杂度为 ()。
A. $O(n \times m)$ B. $O(n + m)$ C. $O(n^2)$ D. $O(m^2)$
9. 无向完全图有 6 个顶点, 则边的数量为 ()。
A. 10 B. 12 C. 15 D. 30
10. 用除留余数法构造散列函数时, 模 p 通常选择 ()。
A. 偶数 B. 2 的幂 C. 质数 D. 10 的倍数

参考答案与解析

答案： 1. B 2. B 3. C 4. B 5. A 6. D 7. B 8. B 9. C
10. C 解析：

1. 数据结构在内存中的表示是存储结构（物理结构），逻辑结构是用户视图。
2. 栈的关键特性是后进先出（LIFO），只允许在栈顶操作。任务调度是队列的应用。
3. 二叉树性质 $n_0 = n_2 + 1 = 10 + 1 = 11$ 。
4. BFS 逐层访问顶点，需要队列来保存待访问的顶点序列。
5. 折半查找要求顺序存储（数组）且有序，才能通过下标快速定位中间位置。
6. 简单选择排序无论数据分布如何，比较次数恒为 $n(n-1)/2$ ，始终 $O(n^2)$ 。
7. 先让 s 指向 p 的后继，再将 p 的 `next` 指向 s ，顺序不能颠倒。
8. KMP 算法利用 `next` 数组避免主串回溯，时间复杂度为 $O(n+m)$ 。
9. 无向完全图边数 $= n(n-1)/2 = 6 \times 5/2 = 15$ 。
10. 除留余数法选择质数作为模数，可以均匀分布关键码，减少冲突。

二、填空题

1. 算法的五个基本特性是：有穷性、确定性、可行性、_____ 和 _____。
2. 在顺序表中插入一个元素，平均需要移动 _____ 个元素。
3. 循环队列判空的条件是 _____，判满的条件是 _____（牺牲一个单元）。
4. 二叉树的第 i 层最多有 _____ 个结点。
5. 图的两种常用存储结构是 _____ 和 _____。
6. 直接插入排序是一种 _____（稳定/不稳定）的排序算法。

参考答案与解析

答案：1. 输入、输出 2. $n/2$ 3. $\text{front} = \text{rear}, (\text{rear} + 1) \% \text{QueueSize} = \text{front}$ 4. 2^{i-1} 5. 邻接矩阵、邻接表 6. 稳定 解析：

1. 算法的五大特性：有穷性、确定性、可行性、输入（零个或多个）、输出（一个或多个）。
2. 顺序表插入平均移动 $n/2$ 个元素，复杂度 $O(n)$ 。
3. 循环队列通过牺牲一个单元来区分队空和队满。
4. 二叉树性质 1：第 i 层最多 2^{i-1} 个结点 ($i \geq 1$)。
5. 图的存储结构主要有邻接矩阵（数组表示）和邻接表（链表表示）。
6. 直接插入排序中，相等元素的相对次序不会改变，因此是稳定排序。

三、简答题

1. 简述顺序存储结构和链式存储结构各自的优缺点。
2. 什么是二叉树的完全二叉树？它有什么性质？
3. 简述 Dijkstra 算法求单源最短路径的基本思想。
4. 什么是哈希冲突？常用的处理方法有哪几种？

参考答案与解析

解析：

1. 顺序存储优点：随机存取 $O(1)$ 、存储密度高；缺点：插入删除需移动元素 $O(n)$ 、需预分配空间。链式存储优点：动态分配、插入删除只需修改指针 $O(1)$ （已知位置）；缺点：顺序存取 $O(n)$ 、存储密度低。
2. 完全二叉树：叶子只出现在最下两层，且最下层的叶子靠左排列。性质：深度为 $\lfloor \log_2 n \rfloor + 1$ ；可用数组高效顺序存储； $n_0 = \lceil n/2 \rceil$ 。
3. Dijkstra 算法：初始化源点距离为 0，其他顶点距离为 ∞ ，已选集合 S 只包含源点。每次从 $V - S$ 中选 dist 最小的顶点加入 S ，然后更新与其相邻顶点的 dist 值。重复直到所有顶点都在 S 中。
4. 哈希冲突指不同关键码通过散列函数映射到同一地址。处理方法：开放定址法（线性探测、二次探测）；链地址法；再散列法。

四、综合应用题

- 已知一棵二叉树的先序遍历序列为 ABCDEFGH，中序遍历序列为 CBEDFAGH。
 - 画出这棵二叉树的结构。
 - 写出该二叉树的后序遍历序列。
 - 计算该二叉树的叶子结点数和深度。
- 设散列表长度为 11，散列函数为 $H(\text{key}) = \text{key} \% 11$ ，采用线性探测再散列处理冲突。依次插入关键码集合 {22, 8, 15, 31, 19, 42, 7}。
 - 画出散列表的存储状态。
 - 计算等概率下查找成功的平均查找长度 ASL。

参考答案与解析

解析：

- 二叉树结构：根为 A，左子树根为 B，右子树根为 G。B 的左孩子为 C（无子树），右孩子为 D，D 的左孩子为 E（无子树），右孩子为 F（无子树）。G 的右孩子为 H（无子树）。
 - 后序遍历序列：CEFDBHGA。
 - 叶子结点：C、E、F、H，共 4 个。深度为 4（A-B-D-F 层）。
- 散列表（下标 0-10）：

下标	0	1	2	3	4	5	6	7	8	9	10
关键码	22				15			7	8	19	42
									31		

插入过程：22%11=0（0 处），8%11=8（8 处），15%11=4（4 处），31%11=9（9 处），19%11=8（冲突，探测到 10），42%11=9（冲突，探测到 10 冲突，探测到 0 冲突，探测到 1），7%11=7（7 处）。

- ASL_{成功} = (1 + 1 + 1 + 1 + 2 + 4 + 1) / 7 = 11 / 7 ≈ 1.57。

第 C 章 考前速记手册

第一章绪论速记

数据结构基础

- [必记] 算法 + 数据结构 = 程序 (Wirth 公式)
- [必记] 逻辑结构四分类: 集合、线性、树、图
- [必记] 算法五大特性: 有穷、确定、可行、输入、输出
- [公式] 常见复杂度: $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n)$
- [易错] 算法 $\neq$ 程序! 程序不满足有穷性时不是算法
- [易错] ADT 不考虑实现细节, 只考虑逻辑结构和操作接口

第二章线性表速记

顺序表 vs 链表

- [必记] 顺序表随机存取 $O(1)$, 链表顺序存取 $O(n)$
- [必记] 链表插入删除 $O(1)$ (已知位置), 顺序表 $O(n)$
- [必记] 顺序表地址是下标的线性函数: $\text{Loc}(a_i) = \text{Loc}(a_0) + i \times c$
- [必记] 头结点的作用: 统一空表和非空表的处理
- [易错] 单链表插入需先找到前驱结点
- [公式] 循环链表判空: $\text{head} \rightarrow \text{next} == \text{head}$
- [公式] 双向链表插入需同时修改 prior 和 next

第三章栈和队列速记

栈（LIFO）与队列（FIFO）

- [必记] 栈：后进先出，仅在栈顶操作（顺序栈 $\text{top}=-1$ 为空）
- [必记] 队列：先进先出，队尾入队、队头出队
- [必记] 循环队列：牺牲一个单元区分队空队满
- [公式] 队空： $\text{front} == \text{rear}$ ；队满： $(\text{rear}+1)\%m == \text{front}$
- [公式] 元素个数： $(\text{rear} - \text{front} + m) \% m$
- [必记] 栈的应用：括号匹配、进制转换、函数调用、表达式求值
- [易错] 链栈入栈用头插法，不需判满

第四章字符串和数组速记

模式匹配与矩阵压缩

- [必记] BF 算法：主串回溯， $O(n \times m)$
- [必记] KMP 算法：主串不回溯，利用 next 数组， $O(n + m)$
- [公式] 按行优先寻址： $\text{Loc}(a_{ij}) = \text{Loc}(a_{l_1 l_2}) + ((i - l_1)(h_2 - l_2 + 1) + (j - l_2)) \times c$
- [必记] 对称矩阵压缩： $n^2 \rightarrow n(n+1)/2$ ， $k = i(i-1)/2 + j - 1$
- [易错] 数组不支持插入和删除操作
- [易错] 矩阵压缩存储后仍支持随机存取

第五章树和二叉树速记

二叉树性质与遍历

- [必记] 二叉树性质: $n_0 = n_2 + 1$
- [必记] 深度 k 的二叉树最多 $2^k - 1$ 个结点, 第 i 层最多 2^{i-1} 个
- [必记] 完全二叉树深度 $\lfloor \log_2 n \rfloor + 1$
- [必记] 遍历: 先序 (根左右)、中序 (左根右)、后序 (左右根)、层序
- [公式] 中序 + 先序/后序可唯一确定二叉树
- [易错] 二叉树不是度为 2 的树!
- [公式] 树转二叉树: 孩子兄弟表示法 (左孩子右兄弟)

第六章图速记

图的存储与算法

- [必记] 邻接矩阵 $O(n^2)$, 邻接表 $O(n + e)$
- [必记] DFS: 类似先序遍历, 用递归或栈
- [必记] BFS: 类似层序遍历, 用队列
- [必记] Prim: 选点, $O(n^2)$, 适合稠密图
- [必记] Kruskal: 选边, $O(e \log e)$, 适合稀疏图
- [必记] Dijkstra: 单源最短路径, $O(n^2)$, 不能处理负权边
- [易错] 拓扑排序仅适用于有向无环图 (DAG)
- [公式] 完全图边数: 无向 $n(n-1)/2$, 有向 $n(n-1)$

第七章查找技术速记

查找算法总结

- [必记]顺序查找 $ASL_{成功} = (n + 1)/2$, $O(n)$
- [必记]折半查找 $O(\log_2 n)$, 要求有序顺序表
- [必记]BST 中序遍历得递增序列
- [公式]散列函数: 除留余数法 $H(key) = key \% p$ (p 为质数)
- [必记]冲突处理: 开放定址法 (线性探测)、链地址法
- [易错]BST 最坏情况 (单支树) 退化为 $O(n)$
- [易错]散列查找仍需比较关键码 (冲突时)

第八章排序技术速记

排序算法对比

- [必记]稳定排序: 直接插入、冒泡、归并、基数
- [必记]不稳定排序: 快速、简单选择、堆
- [公式] $O(n \log n)$: 快速排序 (平均)、堆排序、归并排序
- [公式] $O(n^2)$: 直接插入、冒泡、简单选择
- [易错]快速排序最坏 $O(n^2)$ (基准选得不好时)
- [易错]基数排序不是基于比较的排序
- [公式]归并排序需 $O(n)$ 辅助空间